```
import akka.cluster.Cluster
import akka.actor.typed.scaladsl.{ Behaviors, Routers }
import akka.actor.typed.{ ActorSystem, Behavior }
import com.typesafe.config.ConfigFactory

object CounterWordsApp {

  def main(args: Array[String]): Unit = {
    if (args.isEmpty) {
      startup("aggregator", 0)                                          ❶
    } else {
      require(
        args.size == 2,
        "Usage: two params required 'role' and 'port'")                 ❷
      startup(args(0), args(1).toInt)
    }
  }

  def startup(role: String, port: Int): Unit = {

    val config = ConfigFactory
      .parseString(s"""
      akka.remote.artery.canonical.port=$port
      akka.cluster.roles = [$role]
      """)
      .withFallback(ConfigFactory.load("words"))                        ❸

    ActorSystem[ClusterDomainEvent](ClusteredGuardian(), "WordsCluster",
    ➥config)                                                           ❹

  }

...                                                                     ❺
```

❶ When no params are passed, spawns an aggregator node in a random port

❷ When params are passed, two are required: role and port

❸ Adds role and port to the selected configuration file, words.conf

❹ Starts the cluster

❺ The file continues with the ClusteredGuardian definition.

These parameters can be the following:

```
sbt:chapter08b>runMain example.countwords.CounterApp seed 2551
sbt:chapter08b>runMain example.countwords.CounterApp director 2555
```